

Week 16 Submission Report

Author: Royas Shakya

Course: AI Fellowship 2026

Assignment: Task 3 - Agentify the Assistant

Submission date: 2026-09-13

Project and deliverables

The submission extends the Week 15 assistant with an iterative cross-source verification feature. It includes the complete shared assistant source, the production API and Streamlit frontend, the new agentic loop, context engineering, an updated architecture, and a custom evaluation harness with measured results.

Required deliverable	Submitted artifact
Updated W15 assistant and agentic source	task1-ai-assistant/app; task2-production/backend and frontend
README sections a-c and additional requirements	README.md; docs/ASSESSMENT.pdf
Updated agentic architecture	architecture/agentic-loop.svg; architecture/diagram.md
Evaluation harness and results	task1-ai-assistant/app/evaluation; evaluation/RESULTS.md and raw JSON

The architecture uses one agent, so no multi-agent coordination structure is required. API and Docker instructions are in docs/SETUP.md. Runtime credentials and generated databases are excluded.

Implementation decisions

Royas Shakya · AI Fellowship 2026 · Task 3

This submission extends the W15 RAG assistant with model-directed cross-source verification. A fixed pipeline is insufficient because the next source and search query depend on gaps or disagreements discovered in earlier results.

a. Context Engineering Technique

ContextManager caps each evidence item at 3,000 characters and compacts accumulated findings above 6,000 characters before the next decision. Summaries incorporate prior findings and request preservation of sources and disagreements; error notes are retained separately. Rebuilding the request from the original question and current findings replaces verbose history. Summary output is capped at 2,000 characters; summarizer failure uses truncation. This limits context growth, with the trade-off that detail can be lost. Identical successful tool calls reuse their result within a request.

b. Agentic Pattern

A single agent chooses knowledge search, web search, calculator/date tools, an answer, or clarification. Later searches depend on earlier evidence, so separate specialists and coordination overhead are not justified here. Compaction addresses context saturation; the same model checking its own work remains a self-verification limitation. Requests stop on answer, clarification, unrecovered error, or five decision iterations. Exhaustion reports incomplete verification. Clarification ends the request; the user resubmits with the missing information.

c. Evaluation Harness

The custom Python harness tests ten queries using expected stop states, required successful tools, flat argument schemas and lexical answer rubrics. It measures task completion, tool correctness, decision-trajectory length and tokens. Hard failures are execution failures; soft failures miss a rubric or trajectory criterion; cascading-soft candidates combine a tool failure, later tool actions and an unmet rubric. Human review must establish causality and citation support. JSON preserves answers and trace excerpts. The results report separates real model evaluation from controlled regression tests.

Additional Requirements

1. Skill vs. Agent. A Skill could describe the verification procedure, but requires an execution host to retrieve data and branch on results. This project supplies that host as a bounded loop and exposes the existing retriever through search_knowledge.

answer and ask_user are terminal control actions, not additional agents.

2. Token and Cost Accounting. Totals include every successful decision and compaction response; local generation uses the model tokenizer and Gemini counts reported thinking tokens. Failed calls can have unreported consumption. Offline tokens are synthetic. A multi-agent baseline is not applicable; no unmeasured dollar cost or overhead multiplier is claimed.

3. Failure Injection. Controlled tests inject search exceptions and provider timeouts into the actual loop, check error propagation, exclude failed sources and verify a configured fallback. A separate live injection runner tests how the model responds to unavailable web search and malformed retrieval. The results report identifies which tests ran and their outcomes; scripted behavior is not presented as autonomous live recovery.

4. Tool vs. Agent Boundary. Web search is a bounded DDGS request with at most five results and a ten-second client timeout. Although the service may consult multiple engines internally, the assistant owns the research state and next decision. There is no delegated autonomous goal or agent-to-agent conversation, so this is a tool boundary.

Evaluation Results

Royas Shakya · Week 16 · Evaluation date: 2026-09-13.

Method

The harness is implemented in plain Python without an external evaluation framework. Ten queries span simple, moderate and complex tasks. The live run used gemini-3.6-flash with a 15-second pause per model call and at most five decisions per query. Scores use the submitted query rubrics without post-run relaxation. Source hashes and environment versions are supplied alongside this report.

Completion requires the expected stopping state, every required successful tool call and the query's lexical content checks. Tool correctness checks names and arguments from execution traces. Initial automatic retrieval is context, not a model-selected tool call. Trajectory reasonableness is measured separately against the expected range. These are useful proxies, not proofs of factual accuracy.

Live summary

Metric	Measured result
Queries evaluated	10
Task completion	50% (5/10)
Tool correctness	60% (6/10)
Mean decision iterations	2.4
All completion and trajectory checks	3/10
Prompt tokens	30,436
Completion/thinking tokens	10,926
Total reported tokens	41,362
Duration	523.3 seconds

Token totals include successful decision and compaction responses. They exclude unknown consumption on failed calls, prior runs, credential checks and separate failure-injection runs. No dollar-cost estimate is asserted. This is a single-agent design; a multi-agent coordination baseline is not applicable.

Per-query results

Query ID	Completed	Tools correct	Decisions	Tokens	Stop reason
simple_01	Yes	Yes	3	6,283	model_answered
simple_02	Yes	Yes	2	1,897	model_answered
simple_03	Yes	Yes	2	2,756	model_answered
moderate_01	Yes	Yes	4	10,031	model_answered
moderate_02	Yes	Yes	3	6,233	model_answered
moderate_03	No	Yes	5	13,105	max_iterations
complex_01	No	No	2	1,057	error
complex_02	No	No	1	0	error
complex_03	No	No	1	0	error
complex_04	No	No	1	0	error

Failure analysis

Four hard failures were provider HTTP 429 errors during the complex queries. No quota-triggered key rotation was used. moderate_03 gathered evidence for five decisions but did not answer; it returned an explicit incomplete-verification response. This is a soft failure, not a completed task. Compaction occurred on this query and its tokens are included.

simple_01 and moderate_01 passed task-content/tool checks but exceeded their expected trajectory ranges, so they are also logged as soft failures. Consequently, five completed tasks and seven total failure flags are not contradictory: two completed tasks carry efficiency failures. There were no classified cascading-soft failures in this run; the taxonomy's causal candidates require human trace review.

Manual review and limits

- simple_01: the RAG definition and named techniques align with the supplied guide; the answer includes document references. Three decisions exceeded the expected one-to-two range.
- simple_02: the final response reflects the date/time tool's returned server clock, including 2026-09-13. Timezone presentation remains dependent on the server.
- simple_03: the calculator result and final answer are 51, as expected for 15% of 340.
- moderate_01: the trace shows a knowledge search followed by two different web searches before answering. This demonstrates model-directed cross-source behavior rather than a fixed one-search sequence. The answer links external sources, but broad claims such as tooling being "mandatory" or a universal "current standard" exceed what the saved excerpts establish. Its automatic pass should not be read as independent validation of every claim.
- moderate_02: the answer distinguishes enduring concepts from gaps in the internal AI overview and meets the automatic rubric. External claims were not independently fact-checked exhaustively.
- moderate_03: the bound and compaction functioned, but evidence-sufficiency judgment did not reach an answer in time.
- Complex-query quality remains unmeasured because provider quota prevented completed responses. A successful full rerun requires available quota; waiting between calls does not repair exhausted daily allowance.

Controlled regression and offline harness

The submitted source passes 24/24 regression/API tests, including multi-step decisions, clarification, malformed arguments, duplicate-call reuse, context compaction and token accounting, provider errors, intentional search/timeout failures, native Gemini final-text adaptation, key isolation and production knowledge-tool startup. See regression_tests.txt. These tests use controlled responses and do not prove autonomous model quality.

The offline harness has four deliberate cases: cross-source completion, clarification, timeout and wrong answer. Its 2/4 completion rate is expected because the final two cases are intentional negatives; it logs one hard and one soft failure. Its 60 tokens are synthetic. See evaluation_offline.md and .json.

Intentional failure injection

Controlled regression tests deliberately make a search executor raise an exception and force a provider timeout. They verify error propagation, exclusion of failed sources, explicit uncertainty in the scripted response, and a configured fallback call. This establishes executable failure handling independently of external quota.

The separate live failure-injection results are in `failure_injection_live.md` and `.json`. The timeout fallback case in that runner uses scripted providers and is labeled as such. Any provider-unavailable cases are inconclusive for autonomous recovery, not passing live tests. Review the injection status together with the response excerpts; do not combine controlled and live success rates.

Reproduce

See `../docs/SETUP.md` for commands. The machine-readable live artifact retains every query's answer, stop state, token counts and trace excerpts. Excerpts are capped and do not preserve the full retrieved corpus. These results demonstrate implemented behavior and expose remaining limitations; they do not claim 100% completion or a deployed production system.